

Q1:

In **update()**, the code iterates backward through the episode (Line 35-36) and computes the **discounted future return** at each time step using backward induction:

$$G_t = r_t + \gamma \cdot G_{t+1}$$

This G_t serves as a sample estimate of $Q^{\pi_\theta}(s_t, a_t)$, and each step contributes

$G_t \cdot \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$ to the gradient, which matches (P2) REINFORCE:

$$\nabla_{\theta} V^{\pi_{\theta}}(\mu) = \mathbb{E}_{\tau \sim P_{\mu}^{\pi_{\theta}}} \left[\sum_{t=0}^{\infty} \gamma^t Q^{\pi_{\theta}}(s_t, a_t) \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \right]$$

and (P1) uses the same total trajectory $G(\tau)$ for all time steps, and (P3) uses a stationary (discounted) state distribution $d_{\mu}^{\pi_{\theta}}$, so they are both not the Q1 wants.

Q2:

- a. The code uses a Linear Softmax (Gibbs/Boltzmann) policy:

$$\pi_{\theta}(a | s) = \frac{\exp(s^{\top} \theta_a)}{\sum_a \exp(s^{\top} \theta_a)}$$

self.theta is a (*state_dim* x *action_dim*) matrix, *np.dot(state, self.theta)* computes linear action preferences.

- b. Using **reversed** enables **O(T)** backward induction to compute all G_t :

$$G_{T-1} = r_{T-1}, G_t = r_t + \gamma \cdot G_{t+1}$$

Without **reversed**, computing $G_t = \sum_{k=t}^{T-1} \gamma^{k-t} r_k$ from scratch at each step would cost **O(T²)**.

- c.

Line 41 computes the **score function** $\nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$, which is the gradient of the log-policy with respect to the parameter matrix θ . This is multiplied by G_t in Line 46 to perform the policy gradient update.

Q3: Bug Identification & Improvement

Problem: Line 41 incorrectly computes the gradient of the log-softmax policy.

Analytical Gradient:

$$\nabla_{\theta_j} \log \pi_{\theta}(a | s) = \begin{cases} s(1 - \pi_{\theta}(a | s)) & \text{if } j=a \\ -s\pi_{\theta}(j | s) & \text{if } j \neq a \end{cases}$$

Logical Error: The current code **state - np.dot(state, probs)** computes a vector minus a scalar (the inner product of state and probabilities), which is mathematically incorrect for a softmax update. Furthermore, it fails to update the weights for $j \neq a$.

Correction:

Correct matrix-form update using outer product

grad = np.outer(state, (np.eye(self.action_dim)[action] - probs))